Arbres Binaires de Recherche

1 Définition

Arbre binaire de recherche

Un **arbre binaire de recherche** (ABR) est un arbre binaire qui vérifie, de manière récursive :

- tous les nœuds du sous-arbre **gauche** ont une valeur **strictement inférieure** à celle de la racine ;
- tous les nœuds du sous-arbre **droit** ont une valeur **strictement supérieure** à celle de la racine ;
- les sous-arbres gauche et droit sont eux-mêmes des ABR.

Exemple et contre-exemple



*À gauche : **pas un ABR**. À droite : **un ABR**.*

Dans l'arbre de gauche, chaque nœud, pris avec ses deux fils, *semble* correct : $1 < 5 < 13$ et $2 < 3$. Pourtant 13, 21, 3 et 2 (en violet foncé) violent la règle vis-à-vis de la racine 8 : 13 et 21 sont dans le sous-arbre gauche mais sont plus grands que 8 ; 3 et 2 sont dans le sous-arbre droit mais sont plus petits que 8.

Le parcours infixe d'un ABR

Le parcours **infixe** visite le sous-arbre gauche, puis la racine, puis le sous-arbre droit.

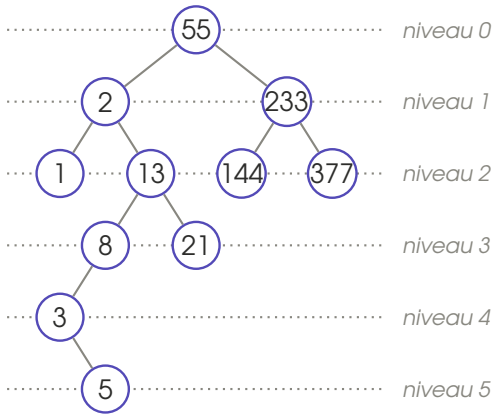
Propriété

Le parcours infixe d'un ABR produit les valeurs dans l'**ordre croissant**.

Pour l'ABR de droite, le parcours infixe donne $1, 2, 3, 5, 8, 13, 21, 34, 55$. Cette propriété fournit un test d'ABR (exercice 2) et un algorithme de tri (exercice 4).

2 Opérations sur les ABR

Toutes les figures de cette section utilisent l'ABR T ci-dessous (11 nœuds, hauteur 5). Son parcours infixe est $1, 2, 3, 5, 8, 13, 21, 55, 144, 233, 377$.

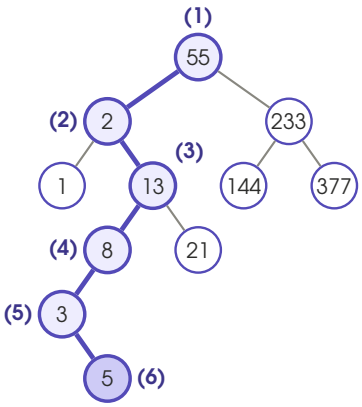


L'ABR T

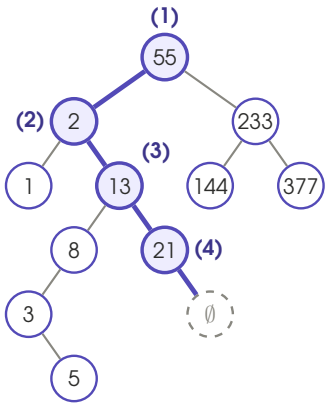
2.1 Recherche d'un élément

L'algorithme s'exécute récursivement. Pour chercher x dans l'arbre a :

- si a est **vide**, x est absent : on renvoie `False` ;
- si x est égal à la valeur de la racine, on renvoie `True` ;
- si x est plus petit, on cherche dans le sous-arbre **gauche** ; s'il est plus grand, dans le sous-arbre **droit**.



recherche de 5 : trouvé en 6 étapes



recherche de 34 : sous-arbre vide, absent

recherche de 5			recherche de 34		
étape	nœud	test	étape	nœud	test
(1)	55	$5 < 55$: gauche	(1)	55	$34 < 55$: gauche
(2)	2	$5 > 2$: droite	(2)	2	$34 > 2$: droite
(3)	13	$5 < 13$: gauche	(3)	13	$34 > 13$: droite
(4)	8	$5 < 8$: gauche	(4)	21	$34 > 21$: droite
(5)	3	$5 > 3$: droite	(5)	$\emptyset$	arbre vide : **absent**
(6)	5	$5 = 5$: **trouvé**			

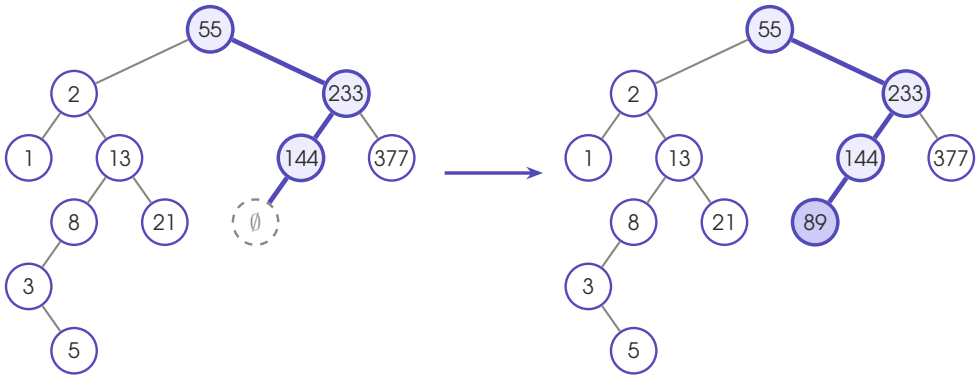
À RETENIR

À chaque comparaison, on **élimine un sous-arbre entier**. On visite au plus un nœud par niveau : au maximum $h + 1$ nœuds, quelle que soit la taille de l'arbre.

2.2 Insertion d'un élément

Pour insérer x dans l'arbre a :

- si a est **vide**, on crée un arbre dont la racine est x et dont les deux sous-arbres sont vides ;
- si x est plus petit que la racine, on insère dans le sous-arbre **gauche** ; s'il est plus grand, dans le sous-arbre **droit** ;
- si x est déjà présent, l'arbre ne change pas (valeurs distinctes).

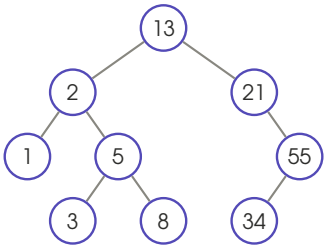


on descend comme pour une recherche de 89

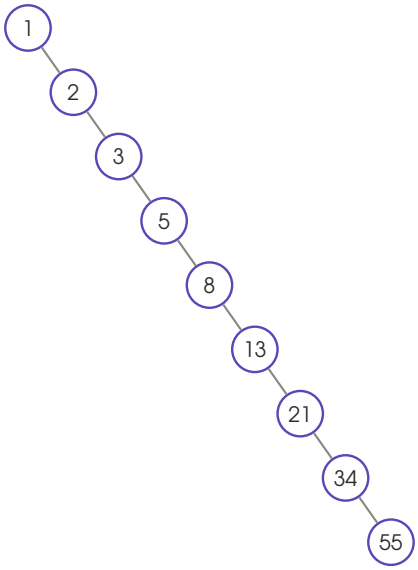
89 devient une feuille

> **À RETENIR**
> Une insertion se fait **toujours en feuille** : on suit exactement le chemin de la recherche, jusqu'à la première place vide. Aucun nœud existant n'est déplacé. La version fonctionnelle reconstruit seulement les nœuds du chemin racine → feuille.

L'ordre d'insertion décide de la forme. Les mêmes neuf valeurs donnent des arbres très différents selon l'ordre dans lequel on les insère :



ordre 13, 2, 21, 1, 5, 55, 3, 8, 34 : $h = 3$



ordre croissant 1, 2, 3, 5, 8, 13, 21, 34, 55 : $h = 8$

Insérer des valeurs déjà triées produit un arbre « en peigne » (dégénéré), qui ressemble à

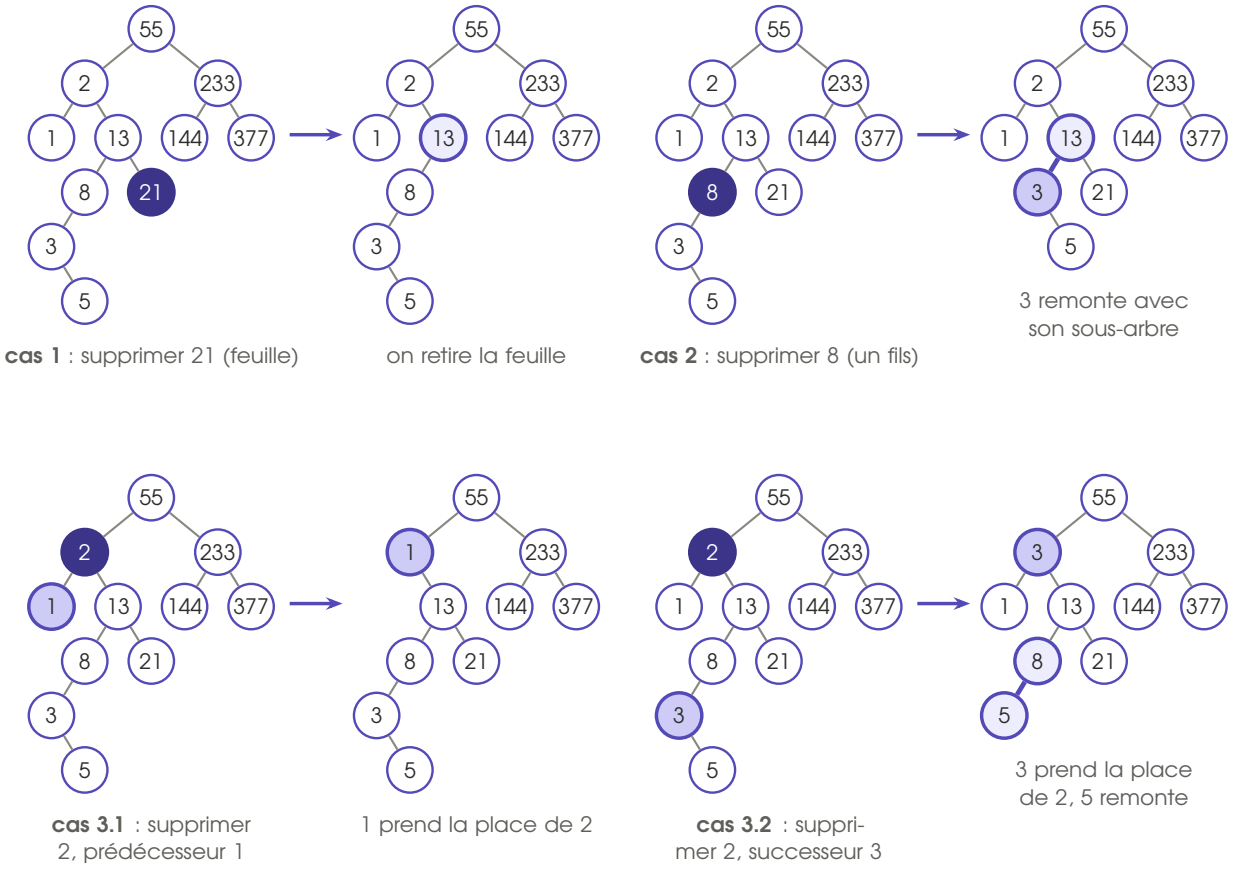
une liste : la recherche n'est alors pas plus rapide qu'un parcours séquentiel.

2.3 Suppression d'un élément

Pour supprimer le nœud contenant x (après l'avoir trouvé par recherche), trois cas :

1. **le nœud est une feuille** : on le retire ;
2. **le nœud a un seul fils** : on le remplace par ce fils (le fils « remonte » avec tout son sous-arbre) ;
3. **le nœud a deux fils** : on le remplace par une valeur qui préserve l'ordre, soit
   - le **prédécesseur** : le maximum du sous-arbre gauche, soit
   - le **successeur** : le minimum du sous-arbre droit.

   Puis on supprime, dans le sous-arbre concerné, le nœud qui vient de monter.



Les trois cas, sur T. En violet foncé : le nœud supprimé ; en violet clair : ce qui change.

2.4 Complexité

Théorème

La recherche, l'insertion et la suppression dans un ABR de hauteur h s'exécutent en $\mathcal{O}(h)$.

Idée de la preuve. Chaque appel récursif descend d'un niveau et effectue un travail constant ; il y a donc au plus $h + 1$ appels. Pour la suppression avec deux fils, la recherche

du successeur prolonge la descente vers le bas : le chemin total reste de longueur au plus h.

Ce théorème ne dit rien de la taille n : tout dépend de la **forme** de l'arbre. Pour n nœuds,

$$\lceil \log_2(n+1) \rceil - 1 \;\leq\; h \;\leq\; n-1.$$

La borne basse est atteinte par un arbre « plein » ; la borne haute par l'arbre « en peigne ».

Tout l'enjeu est de **garder** h **petite**.

3 Exercices

Exercice 1 — ● ○ ○ Arbres binaires quelconques

Ici l'arbre n'est *pas* ordonné : aucune propriété d'ABR ne peut être utilisée.

1. Écrire une fonction `taille(arbre)` qui renvoie le nombre de nœuds d'un arbre binaire.
2. Écrire une fonction `max_arbre(arbre)` qui renvoie la valeur maximale d'un arbre binaire non vide.
3. Écrire une fonction `recherche_arbre(arbre, x)` qui indique si x appartient ou non à un arbre binaire.
4. Quel est le coût de `recherche_arbre` dans le pire cas ? Comparer avec la recherche dans un ABR.

Exercice 2 — ● ● ○ Reconnaître et chercher dans un ABR

1. Écrire une fonction `est_abr(arbre)` qui vérifie qu'un arbre binaire est un ABR.
2. Écrire une fonction `max_abr(arbre)` qui renvoie la valeur maximale d'un ABR non vide, *sans parcourir tout l'arbre*.
3. Écrire une fonction `recherche_abr(arbre, x)` qui indique si x appartient ou non à un ABR.
4. Réécrire `recherche_abr` sans récursivité (boucle `while`).

Exercice 3 — ● ● ○ Insertion et suppression

1. Écrire une fonction récursive `insertion(abr, v)` qui insère `v` dans l'ABR `abr`. La fonction renvoie un nouvel arbre contenant la valeur insérée.
2. Écrire une fonction récursive `suppression(abr, v)` qui supprime `v` de l'ABR `abr`. La fonction renvoie l'arbre modifié.

Indication

Pour la suppression, écrire d'abord une fonction `minimum(abr)` ; traiter ensuite séparément : valeur différente de la racine (descente), puis les cas 1, 2, 3.

Exercice 4 — ● ● ● Aller plus loin : construire et trier

1. Écrire `hauteur(arbre)` en respectant la convention $h(\text{vide}) = -1$.
2. Écrire `construire(valeurs)` qui renvoie l'ABR obtenu en insérant successivement les valeurs d'une liste dans un arbre initialement vide.
3. En déduire `tri_abr(liste)` qui trie une liste de valeurs distinctes en utilisant le parcours infixe.
4. Donner la complexité de `tri_abr` dans le meilleur et dans le pire cas.

4 Corrigés

Exercice 1

```
def taille(a):
    if a == []:
        return 0
    v, g, d = a
    return 1 + taille(g) + taille(d)

def max_arbre(a):                       # arbre non vide, non ordonné
    v, g, d = a
    m = v
    if g != []:
        m = max(m, max_arbre(g))
    if d != []:
        m = max(m, max_arbre(d))
    return m

def recherche_arbre(a, x):
    if a == []:
        return False
    v, g, d = a
    return v == x or recherche_arbre(g, x) or recherche_arbre(d, x)
```

Question 4. Comme rien n'est ordonné, il faut potentiellement visiter tous les nœuds : $\mathcal{O}(n)$. Dans un ABR, chaque comparaison élimine un sous-arbre, d'où $\mathcal{O}(h)$: c'est tout l'intérêt de la structure.

Exercice 2

Question 1, méthode (1) : parcours infixe.

```
def infixe(a):
    if a == []:
        return []
    v, g, d = a
    return infixe(g) + [v] + infixe(d)

def est_abr_infixe(a):                  # valeurs distinctes
    l = infixe(a)
    return all(l[i] < l[i + 1] for i in range(len(l) - 1))
```

Question 1, méthode (2) : bornes. Chaque valeur doit se trouver dans un intervalle ouvert dont les bornes viennent des ancêtres : en descendant à gauche, la borne haute devient la valeur du nœud ; en descendant à droite, c'est la borne basse.

```
def est_abr(a, bas=float('-inf'), haut=float('inf')):
    if a == []:
        return True
    v, g, d = a
```

```
    if not (bas < v < haut):           # v doit être dans l'intervalle ]bas, haut[
        return False
    return est_abr(g, bas, v) and est_abr(d, v, haut)
```

La méthode (2) est en $\mathcal{O}(n)$, comme (1), mais sans construire de liste.

Questions 2, 3 et 4.

```
def max_abr(a):                        # arbre non vide : tout à droite
    v, g, d = a
    return v if d == [] else max_abr(d)

def recherche_abr(a, x):
    if a == []:
        return False                   # arbre vide : x est absent
    v, g, d = a
    if x == v:
        return True
    if x < v:
        return recherche_abr(g, x)     # plus petit : à gauche
    return recherche_abr(d, x)         # plus grand : à droite

def recherche_abr_iter(a, x):          # version itérative
    while a != []:
        v, g, d = a
        if x == v:
            return True
        a = g if x < v else d
    return False
```

Le maximum est le nœud le plus à droite : on descend au plus h fois, soit $\mathcal{O}(h)$.

Exercice 3

```
def insertion(a, x):
    if a == []:
        return [x, [], []]             # place vide : nouvelle feuille
    v, g, d = a
    if x < v:
        return [v, insertion(g, x), d]
    if x > v:
        return [v, g, insertion(d, x)]
    return a                           # déjà présent : rien à faire

def minimum(a):
    v, g, d = a
    return v if g == [] else minimum(g)

def suppression(a, x):
    if a == []:
        return []                      # x absent
    v, g, d = a
```

```
    if x < v:
        return [v, suppression(g, x), d]
    if x > v:
        return [v, g, suppression(d, x)]
    # cas d'égalité
    if g == []:                          # pas de fils gauche
        return d
    if d == []:                          # pas de fils droit
        return g
    m = minimum(d)                       # deux fils : cas de successeur
    return [m, g, suppression(d, m)]
```

Le cas `g == []` traite à la fois la feuille (on renvoie `d`, qui vaut `[]`) et le fils unique droit ; le cas `d == []` traite le fils unique gauche. Pour deux fils, on remonte le minimum du sous-arbre droit puis on le supprime de ce sous-arbre.

Exercice 4

```
def hauteur(a):
    if a == []:
        return -1                        # convention : arbre vide = -1
    v, g, d = a
    return 1 + max(hauteur(g), hauteur(d))
```

```
from functools import reduce

def construire(valeurs):
    return reduce(insertion, valeurs, [])

def tri_abr(liste):
    return infixe(construire(liste))
```

Question 4. Meilleur cas (arbre équilibré) : n insertions de coût $\mathcal{O}(\log n)$, soit $\mathcal{O}(n \log n)$. Pire cas (liste déjà triée, arbre en peigne) : $1 + 2 + \cdots + n$ comparaisons, soit $\mathcal{O}(n^2)$. Dans les deux cas, le parcours infixe coûte $\mathcal{O}(n)$.